Chapter 38: Angular CDK

1. Overview

The Angular Component Dev Kit (`@angular/cdk`) is a set of **behavior primitives** extracted from Angular Material. When the Material team built components like `mat-select`, `mat-tooltip`, `mat-table`, and `mat-sidenav`, they noticed the *hard problems* — positioning a floating panel relative to a trigger, trapping focus inside a modal, announcing changes to screen readers, virtualizing a long list, making an item draggable and sortable — were completely independent of what the component *looked like*. The CDK is that extracted, unstyled, unopinionated layer.

The core philosophy: headless UI. CDK ships zero (or near-zero) CSS and zero visual design. It gives you directives and services that solve *behavior* and *accessibility* correctly, and you decide the DOM structure and styling. This is the same idea as Radix UI / Headless UI in the React world, just older — Angular CDK predates most of those by several years.

Why this matters for interviews: recruiters ask CDK questions to check whether you understand **the difference between "a UI library" and "a UI toolkit"**, and whether you've had to build a custom, accessible component (tooltip, autocomplete, modal, mega-dropdown) without pulling in the entirety of Material's opinionated design.

CDK is organized into independent modules, each importable on its own:

Module	Import	Solves
Overlay	`@angular/cdk/overlay`	Floating UI: positioning, scrolling, backdrop, z-index stacking
Portal	`@angular/cdk/portal`	Rendering content outside its logical template location
Drag and Drop	`@angular/cdk/drag-drop`	Draggable elements, sortable lists, cross-list transfer
Scrolling (Virtual Scroll)	`@angular/cdk/scrolling`	Rendering only visible DOM nodes for huge lists
A11y	`@angular/cdk/a11y`	Focus trapping, focus monitoring, live announcements, high-contrast detection
Layout	`@angular/cdk/layout`	Responsive breakpoint observation (wraps `matchMedia`)
Table	`@angular/cdk/table`	Data-table rendering model (rows/columns) without any styling
Bidi	`@angular/cdk/bidi`	LTR/RTL direction propagation
Collections	`@angular/cdk/collections`	`SelectionModel`, `DataSource` abstraction

Module	Import	Solves
Coercion	`@angular/cdk/coercion`	`coerceBooleanProperty`, `coerceNumberProperty`, etc.

Angular Material is built **on top of** CDK: `mat-menu` and `mat-autocomplete` use `CdkOverlay`/Portal, `mat-table` wraps `CdkTable`, `mat-tree`'s virtual scroll variant uses `CdkVirtualScrollViewport`, and every Material dialog/tooltip uses `FocusTrap`/`LiveAnnouncer` internally. You can use CDK **without** Material at all — that's the entire point.

---

2. Core Concepts

2.1 Overlay (`CdkOverlay`)

The Overlay package is a low-level API for opening floating panels of content on top of the application — tooltips, menus, dropdowns, dialogs, autocomplete panels. It solves problems that are surprisingly hard to get right by hand:

- Rendering the panel **outside** the normal DOM flow so it isn't clipped by `overflow: hidden` ancestors.
- Positioning it relative to a trigger element (or a point, or the viewport) and re-positioning it as the trigger scrolls or the window resizes.
- Managing a shared **z-index stacking context** so overlays never fight each other.
- Providing a **backdrop** (for modal-like behavior, including click-to-close).
- Deciding what happens to the overlay when the page scrolls (`ScrollStrategy`).

Key pieces:

- `Overlay` (injectable service) — the factory. `overlay.create(config)` returns an `OverlayRef`.
- `OverlayConfig` — width/height, `hasBackdrop`, `backdropClass`, `panelClass`, `positionStrategy`, `scrollStrategy`.
- `OverlayRef` — handle to the created overlay. Has `.attach(portal)`, `.detach()`, `.dispose()`, `.backdropClick()` (Observable), `.keydownEvents()`, `.overlayElement`, `.updatePosition()`.
- `OverlayPositionBuilder` — builds position strategies.

Positioning strategies (`PositionStrategy` interface — `.apply()` on attach, `.dispose()` on cleanup):

1. `FlexibleConnectedPositionStrategy` (the one you'll use 95% of the time). Created via `overlay.position().flexibleConnectedTo(elementOrRef)`. You supply a list of preferred `ConnectedPosition` objects (origin X/Y + overlay X/Y + optional offsets), and the strategy tries them in order, falling back to the next one if the overlay would overflow the viewport. It can also **push** the overlay back into the viewport, or **flip** it to the opposite side. This is what powers `mat-menu` (which flips above the trigger if there's no room below) and `mat-autocomplete`.
2. `GlobalPositionStrategy` — positions relative to the viewport itself (e.g., "centered", "top: 0, right: 0"), not relative to any origin element. Used for full-screen/centered dialogs.
3. **(Legacy)** `ConnectedPositionStrategy` — predecessor to flexible-connected; deprecated in favor of it.

Scroll strategies (`ScrollStrategy` — what to do to the overlay when an ancestor scrolls):

- `noop()` — do nothing; overlay stays fixed in the viewport (or visually detaches from its trigger). Rarely correct for connected overlays.
- `block()` — blocks page scrolling entirely while the overlay is open (typical for modal dialogs).
- `reposition()` — recomputes the overlay's position on every scroll event, keeping it visually anchored to the trigger (typical for tooltips/menus/autocomplete). Can be throttled via `{ scrollThrottle: ms }`.
- `close()` — closes the overlay as soon as scrolling starts (used by some dropdown-style pickers).

You get these from `overlay.scrollStrategies.reposition()` etc.

2.2 Portal

A **Portal** is a piece of UI content that isn't rendered where it's declared — it's a "recipe" for content that can be attached to a **PortalOutlet** (the CDK term for a portal "host"/target) anywhere in the DOM, including outside the current component's view. `OverlayRef` implements `PortalOutlet`.

Three portal types:

- `ComponentPortal<T>` — wraps a component type (not an instance). Attaching it creates the component via `ComponentPortal` + `ViewContainerRef`/injector, inserts it into the outlet, and returns the `ComponentRef`. Used when the overlay content is itself a full Angular component (e.g., a custom `<app-tooltip>` component).
- `TemplatePortal` — wraps a `TemplateRef` + `ViewContainerRef` (and optional context object). Lets you use an inline `<ng-template>` from the calling template as overlay content — very common for directives like a custom `*cdkConnectedOverlay`-style tooltip where the caller supplies the template.
- `DomPortal` — wraps a raw DOM node / element reference directly, for cases with no Angular component or template involved (e.g. moving an existing native element).

`CdkPortal`, `CdkPortalOutlet` (a.k.a. `cdkPortalOutlet` / legacy `PortalHostDirective`) are the directive forms for wiring portals declaratively in templates without hand-writing TypeScript instantiation.

2.3 Drag and Drop (`CdkDrag` / `CdkDropList`)

- `cdkDrag` — makes any element draggable via pointer/touch/mouse events, using CSS transforms (not repositioning via top/left), so it's GPU-accelerated and doesn't disturb layout of siblings while dragging.
- `cdkDropList` — marks a container as a valid drop zone. Dragged items sort automatically within the list as you drag over siblings (an in-list reorder), and can be configured to accept drags from other lists via `cdkDropListConnectedTo`.
- `(cdkDropListDropped)` emits a `CdkDragDrop<T>` event with `previousIndex` / `currentIndex` / `previousContainer` / `container` / `item`; you're expected to mutate your own data array (commonly with the helper `moveItemInArray` / `transferArrayItem` from `@angular/cdk/drag-drop`) — **CDK does not mutate your data model for you**.
- `cdkDragHandle` — restricts the draggable "grab" area to a sub-element (e.g., a handle icon) instead of the whole row.
- `cdkDragPreview` / `cdkDragPlaceholder` — customize the floating preview shown while dragging and the placeholder left in the original slot.

- Boundaries: `cdkDragBoundary`, axis lock: `cdkDragLockAxis` (`'x'` | `'y'`), free dragging without a list: `cdkDrag` alone (no `cdkDropList` parent) gives free-form draggable elements (e.g., draggable dialogs).
- Sorting is achieved by CDK computing sibling positions and animating displaced items with CSS transforms as the pointer moves — no full DOM re-render during drag.

2.4 Virtual Scrolling (`CdkVirtualScrollViewport`)

`cdk-virtual-scroll-viewport` renders **only the DOM nodes currently visible (plus a small buffer)**, regardless of how large the backing array is. This is essential for lists of thousands+ items — rendering 10,000 real rows would devastate layout/paint performance and memory.

- `<cdk-virtual-scroll-viewport [itemSize]="50">` + `*cdkVirtualFor="let item of items"` (a virtualization-aware analog of `*ngFor` / `@for`).
- `itemSize` (pixels) is required by the default `FixedSizeVirtualScrollStrategy` — it assumes **every row is exactly this height**. This lets it do O(1) math (`scrollTop / itemSize`) to know which index is at the top, rather than measuring the DOM.
- Custom strategies: implement `VirtualScrollStrategy` for variable-height rows (more expensive — requires measuring or estimating).
- `viewport.scrollToIndex(i)`, `viewport.scrollToOffset(px)`, `viewport.getRenderedRange()`, `viewport.elementScrolled()` are commonly used APIs.
- Buffer configuration: `minBufferPx` / `maxBufferPx` control how much extra content is rendered outside the visible viewport, trading smoothness of fast scrolling against extra DOM nodes.
- CDK table has a virtual-scroll-friendly integration too (`cdk-table` inside a viewport), and Material's `mat-table` + `cdk-virtual-scroll-viewport` combo is a common production pattern.

2.5 A11y Module

- `FocusTrap` (`cdkTrapFocus` directive, or `FocusTrapFactory` service) — confines Tab/Shift+Tab cycling to within an element, so keyboard focus can't leak out to the rest of the page. Essential for modals/dialogs. `cdkTrapFocusAutoCapture` additionally moves focus into the trap immediately on attach and can restore it to the previously focused element on destroy.
- `LiveAnnouncer` — injectable service (`announcer.announce('Item deleted', 'polite')`) that writes text into an offscreen `aria-live` region so screen readers announce dynamic changes that wouldn't otherwise be noticed (no route/DOM focus change occurred). Politeness levels: `'polite'` (default, waits for a pause) vs `'assertive'` (interrupts immediately).
- `FocusMonitor` (`cdkMonitorFocus` / `cdkMonitorSubtreeFocus` directives or the `FocusMonitor` service) — tracks *how* an element was focused: `'mouse'`, `'keyboard'`, `'touch'`, `'program'` (via `.focus()` call), or `null` (not focused). This is how Material shows a focus ring only for keyboard navigation, not for mouse clicks — a proper implementation of `:focus-visible`-style behavior before that pseudo-class was reliable across browsers. `focusMonitor.monitor(el)` returns an Observable of origin changes.
- `InteractivityChecker` — utility used internally to determine if an element is currently focusable/tabbable (checks `disabled`, visibility, `tabindex`, native tag semantics).
- `AriaDescriber` — attaches/removes descriptive text via an offscreen element referenced by `aria-describedby`, without needing a visible tooltip DOM element to exist for the description to work.

- `HighContrastModeDetector` — detects OS/browser high-contrast mode and adds a class to `<body>` (`cdk-high-contrast-active`, plus black-on-white / white-on-black variants) so components can adjust styling.

2.6 Layout Module (`BreakpointObserver`)

- `BreakpointObserver.observe(query | query[])` returns an `Observable<BreakpointState>` (`{ matches: boolean, breakpoints: {...} }`) that emits whenever any of the given media queries' match state changes. It's a thin, Angular-idiomatic, multiple-query-friendly wrapper around `window.matchMedia`, integrated with Angular's zone/change detection so results propagate correctly.
- `Breakpoints` — a predefined constants object with common breakpoint queries (`Breakpoints.Handset`, `Breakpoints.Tablet`, `Breakpoints.Web`, `Breakpoints.HandsetPortrait`, etc.), based on Material's breakpoint system, so you don't hand-write media query strings.
- `BreakpointObserver.isMatched(query)` — synchronous, one-off check.
- This is the standard way to build responsive *logic* (not just responsive CSS) in Angular — e.g., switching a `mat-sidenav-mode` between `'over'` and `'side'`, or switching a layout component entirely, based on viewport size.

2.7 Table (`CdkTable`)

- `CdkTable` provides the **data-binding and row/column rendering model** for tabular data with zero built-in styling — no borders, no zebra striping, no Material Design look. You provide the CSS.
- Column definitions (`cdkColumnDef` + `cdkHeaderCellDef` / `cdkCellDef`) are declared independently of row order, then rows (`cdkHeaderRowDef`, `cdkRowDef`) declare which columns to render in what order — this indirection lets you reuse column definitions across differently-shaped rows, and reorder columns without touching the column templates.
- Accepts a `DataSource<T>` (an abstract class with `connect()` / `disconnect()`), which can be:
  - a plain array (auto-wrapped),
  - an `Observable<T[]>`,
  - or a custom class connecting to a paginated/sorted/filtered/streamed backend — this is the extension point for server-side pagination/sorting.
- `mat-table` is literally `CdkTable` with Material's default styling, sticky-header CSS, and directive selectors (`mat-header-cell` instead of `cdk-header-cell`) layered on top — internally `MatTable extends CdkTable`.
- Supports sticky rows/columns (`sticky`, `stickyEnd` on column/row defs) and works with `cdk-virtual-scroll-viewport` for large datasets.

2.8 CDK vs. Angular Material — the essential distinction

	Angular CDK	**Angular Material**
Provides	Behavior, accessibility, positioning logic	Fully styled, opinionated Material-Design components

	Angular CDK	Angular Material
CSS	Minimal/none (only what's structurally required, e.g., `cdk-overlay-container` positioning)	Full Material theme, typography, elevation, ripples
Use case	Building a **custom-look** component that needs correct, accessible behavior	Building an app that wants Material's look-and-feel out of the box
Dependency direction	Standalone — Material depends on CDK, not vice versa	Depends on CDK internally
Interview framing	"Headless"/"unstyled" primitives, like Radix/Headless UI	"Batteries-included" component library

You reach for CDK directly whenever you need Material-grade *behavior* correctness (overlay positioning, focus trapping, virtualization, drag-drop) but a bespoke design system/branding that Material's styling would fight against.

---

3. Code Examples

3.1 Custom overlay-based tooltip/dropdown using CdkOverlay + Portal

```
import {
  Directive, Input, TemplateRef, ViewContainerRef, ElementRef,
  OnDestroy, HostListener, Injector
} from '@angular/core';
import {
  Overlay, OverlayRef, FlexibleConnectedPositionStrategy, ConnectedPosition
} from '@angular/cdk/overlay';
import { TemplatePortal } from '@angular/cdk/portal';

@Directive({
  selector: '[appTooltip]',
  standalone: true,
})
export class TooltipDirective implements OnDestroy {
  @Input('appTooltip') template!: TemplateRef<unknown>;

  private overlayRef: OverlayRef | null = null;

  private readonly positions: ConnectedPosition[] = [
    { originX: 'center', originY: 'top', overlayX: 'center', overlayY: 'bottom', offsetY: -8
},
    { originX: 'center', originY: 'bottom', overlayX: 'center', overlayY: 'top', offsetY: 8
},
  ];

  constructor(
```

```
    private overlay: Overlay,
    private elementRef: ElementRef<HTMLElement>,
    private viewContainerRef: ViewContainerRef,
  ) {}

  @HostListener('mouseenter')
  show(): void {
    if (this.overlayRef?.hasAttached()) return;

    const positionStrategy: FlexibleConnectedPositionStrategy = this.overlay
      .position()
      .flexibleConnectedTo(this.elementRef)
      .withPositions(this.positions)
      .withPush(true) // push back into viewport instead of overflowing
      .withFlexibleDimensions(false);

    this.overlayRef = this.overlay.create({
      positionStrategy,
      scrollStrategy: this.overlay.scrollStrategies.reposition({ scrollThrottle: 20 }),
      hasBackdrop: false,
      panelClass: 'app-tooltip-panel',
    });

    const portal = new TemplatePortal(this.template, this.viewContainerRef);
    this.overlayRef.attach(portal);
  }

  @HostListener('mouseleave')
  hide(): void {
    this.overlayRef?.detach();
  }

  ngOnDestroy(): void {
    this.overlayRef?.dispose();
  }
}
```

Usage:

```
<button [appTooltip]="tipTpl">Hover me</button>
<ng-template #tipTpl>
  <div class="my-tooltip">Custom styled tooltip content, fully yours.</div>
</ng-template>
```

A `ComponentPortal` variant (for a full standalone dropdown component instead of an inline template) looks like:

```
import { ComponentPortal } from '@angular/cdk/portal';
import { Overlay } from '@angular/cdk/overlay';
import { Injector, Component } from '@angular/core';

export function openDropdown(overlay: Overlay, origin: HTMLElement, injector: Injector) {
```

```
  const overlayRef = overlay.create({
    positionStrategy: overlay.position().flexibleConnectedTo(origin)
      .withPositions([{ originX: 'start', originY: 'bottom', overlayX: 'start', overlayY:
'top' }]),
    scrollStrategy: overlay.scrollStrategies.close(),
    hasBackdrop: true,
    backdropClass: 'cdk-overlay-transparent-backdrop',
  });

  const portal = new ComponentPortal(DropdownPanelComponent, null, injector);
  const componentRef = overlayRef.attach(portal);

  overlayRef.backdropClick().subscribe(() => overlayRef.dispose());

  return { overlayRef, componentRef };
}
```

3.2 Reorderable list with `CdkDrag` / `CdkDropList`

```
import { Component } from '@angular/core';
import { CdkDropList, CdkDrag, CdkDragDrop, moveItemInArray } from '@angular/cdk/drag-drop';

@Component({
  selector: 'app-task-list',
  standalone: true,
  imports: [CdkDropList, CdkDrag],
  template: `
    <div cdkDropList class="task-list" (cdkDropListDropped)="drop($event)">
      <div class="task-box" *ngFor="let task of tasks" cdkDrag>
        <span cdkDragHandle class="handle">☰</span>
        {{ task }}
      </div>
    </div>
  `,
  styles: [`
    .task-list { width: 300px; }
    .task-box { padding: 8px 12px; border: 1px solid #ccc; margin-bottom: 4px; background:
white; }
    .cdk-drag-preview { box-shadow: 0 4px 10px rgba(0,0,0,.3); }
    .cdk-drag-placeholder { opacity: 0.3; }
    .cdk-drop-list-dragging .task-box:not(.cdk-drag-placeholder) { transition: transform
200ms ease; }
  `],
})
export class TaskListComponent {
  tasks = ['Design overlay', 'Wire drag-drop', 'Add virtual scroll', 'Ship it'];

  drop(event: CdkDragDrop<string[]>): void {
    moveItemInArray(this.tasks, event.previousIndex, event.currentIndex);
  }
}
```

Cross-list transfer (e.g., a Kanban board):

```
import { transferArrayItem, moveItemInArray, CdkDragDrop } from '@angular/cdk/drag-drop';

drop(event: CdkDragDrop<string[]>): void {
  if (event.previousContainer === event.container) {
    moveItemInArray(event.container.data, event.previousIndex, event.currentIndex);
  } else {
    transferArrayItem(
      event.previousContainer.data,
      event.container.data,
      event.previousIndex,
      event.currentIndex,
    );
  }
}
```

```
<div cdkDropList id="todo" [cdkDropListConnectedTo]="['done']"
     [cdkDropListData]="todoItems" (cdkDropListDropped)="drop($event)">
  <div class="item" *ngFor="let item of todoItems" cdkDrag>{{ item }}</div>
</div>
<div cdkDropList id="done" [cdkDropListConnectedTo]="['todo']"
     [cdkDropListData]="doneItems" (cdkDropListDropped)="drop($event)">
  <div class="item" *ngFor="let item of doneItems" cdkDrag>{{ item }}</div>
</div>
```

3.3 `CdkVirtualScrollViewport` for a large list

```
import { Component } from '@angular/core';
import { ScrollingModule } from '@angular/cdk/scrolling';

@Component({
  selector: 'app-big-list',
  standalone: true,
  imports: [ScrollingModule],
  template: `
    <cdk-virtual-scroll-viewport itemSize="48" class="viewport">
      <div *cdkVirtualFor="let item of items; let i = index" class="row">
        #{{ i }} — {{ item }}
      </div>
    </cdk-virtual-scroll-viewport>
  `,
  styles: [`
    .viewport { height: 480px; width: 100%; border: 1px solid #ddd; }
    .row { height: 48px; display: flex; align-items: center; padding: 0 12px; }
  `],
})
export class BigListComponent {
  items = Array.from({ length: 100_000 }, (_, i) => `Row item ${i}`);
}
```

Only ~15-20 `.row` DOM nodes ever exist at once, regardless of the 100,000-length array — `itemSize="48"` tells the default strategy every row is 48px so it can compute the visible index range with pure arithmetic instead of measuring the DOM.

3.4 `BreakpointObserver` responsive service

```
import { Injectable } from '@angular/core';
import { BreakpointObserver, Breakpoints, BreakpointState } from '@angular/cdk/layout';
import { Observable, map, shareReplay } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class ResponsiveService {
  private readonly breakpoint$: Observable<BreakpointState> =
this.breakpointObserver.observe([
    Breakpoints.HandsetPortrait,
    Breakpoints.TabletPortrait,
    Breakpoints.Web,
  ]);

  readonly isHandset$ = this.breakpointObserver
    .observe(Breakpoints.Handset)
    .pipe(map(result => result.matches), shareReplay({ bufferSize: 1, refCount: true }));

  readonly sidenavMode$ = this.isHandset$.pipe(
    map(isHandset => (isHandset ? 'over' : 'side') as 'over' | 'side'),
  );

  constructor(private breakpointObserver: BreakpointObserver) {}
}
```

```
// Component usage
@Component({ /* ... */ })
export class ShellComponent {
  readonly responsive = inject(ResponsiveService);
}
```

```
<mat-sidenav-container>
  <mat-sidenav [mode]="(responsive.sidenavMode$ | async)!" opened>...</mat-sidenav>
</mat-sidenav-container>
```

---

4. Internal Working

4.1 How `CdkOverlay` creates and manages the overlay container

1. **Lazy singleton container.** The first time any overlay is created in the app, `OverlayContainer` (an injectable service) lazily creates a single `<div class="cdk-overlay-container">` and appends it as the **last child of** `<body>`. This is deliberate: being the last element in `<body>`, combined with a high `z-index` on `.cdk-overlay-container` (and its own stacking context), guarantees overlays paint above

ordinary app content without every component needing to manually manage z-index. All overlays in the app share this one container, each becoming its own `<div class="cdk-overlay-pane">` inside it.

2. **Escaping ancestor clipping.** Because the pane lives at the `<body>` level rather than nested inside the triggering component's DOM subtree, it is immune to `overflow: hidden` / `overflow: auto` or `transform` contexts on ancestors that would otherwise clip or reposition it (a classic bug with naively-implemented dropdowns nested in scrollable cards/tables).
3. **Stacking order** among multiple simultaneously-open overlays (e.g., a menu that opens a nested submenu) is maintained simply by DOM append order combined with a shared, incrementing internal z-index baseline CDK applies via CSS classes — later-created overlay panes sit later in the container and thus above earlier ones by default painting order; backdrops get a z-index just below their owning pane.
4. **Position strategy lifecycle.** `overlay.create(config)` builds an `OverlayRef` holding the configured `PositionStrategy` and `ScrollStrategy` but does **not** position anything yet. On `overlayRef.attach(portal)`: the portal's content is instantiated into the pane element, then the position strategy's `.apply()` runs, measuring the origin element's `getBoundingClientRect()` and the overlay pane's own measured size, computing the best-fit position from the candidate list (`FlexibleConnectedPositionStrategy` tries each `ConnectedPosition` in order, checking if it fits within the viewport, using `push` / `flip` fallback logic), and finally sets inline `transform` / `top` / `left` styles on the pane.
5. **Keeping it anchored.** The chosen `ScrollStrategy` is enabled at the same time: `reposition()` subscribes to a shared, throttled scroll-event dispatcher (`ScrollDispatcher`, itself listening on all scrollable ancestors found via `ScrollingModule`'s `cdkScrollable` directive registrations) and calls `overlayRef.updatePosition()` on each tick, which re-runs the position strategy's fit calculation and updates the transform. `block()` instead sets `overflow: hidden` styles on `html` / `body` (and compensates for scrollbar-width) so nothing can scroll while the overlay is open — it does not track position because nothing moves.
6. **Resize** is handled similarly via a `ViewportRuler` service that listens to `window.resize` (debounced), triggering another `updatePosition()` pass.
7. **Teardown.** `overlayRef.dispose()` detaches the portal (destroying the component/embedded view via its `ViewContainerRef` / `ComponentFactoryResolver` machinery), removes the pane and backdrop DOM nodes, unsubscribes the scroll/resize listeners, and disposes the position strategy. The shared `cdk-overlay-container` div itself is left in place for reuse by future overlays (or removed if `OverlayContainer.ngOnDestroy` runs, e.g. in tests via `TestBed`).

4.2 How `CdkVirtualScrollViewport` only renders visible items

1. The viewport element listens to its own `scroll` events (via `ViewportRuler` / `ScrollDispatcher` infrastructure shared with Overlay) and, on each scroll tick, asks its configured `VirtualScrollStrategy` to recompute the **rendered range**.
2. The default `FixedSizeVirtualScrollStrategy` does this with pure arithmetic, not DOM measurement: given `scrollTop`, `itemSize`, and the viewport's own clientHeight, it computes `startIndex = Math.floor(scrollTop / itemSize)` and an `endIndex` covering the visible height, then pads that range outward by `minBufferPx` / `maxBufferPx` worth of extra items so fast scrolling doesn't show blank flashes before new rows render.
3. `*cdkVirtualFor` (analogous to `*ngFor` but virtualization-aware) is told the computed range and only

instantiates embedded views for indices within that range, recycling/destroying views that scroll out and creating new ones for indices that scroll in — it does not keep 100,000 hidden `<div>`s in the DOM, it only ever holds the small rendered-range count.

4. To make the scrollbar behave correctly (proportionate thumb size, correct total scrollable height) despite only a handful of real rows existing, the viewport creates an internal "total content size" spacer/sizing element sized to `itemSize * totalItemCount`, so the browser's native scrollbar reflects the *virtual* full length even though the actual rendered content is tiny. As `scrollTop` changes, the rendered rows are translated (via CSS transform, same trick as drag-and-drop) to the correct visual offset within that virtual space rather than being re-parented at different DOM y-coordinates.
5. This is why `itemSize` must be accurate for the default strategy — the index-from-scrollTop math is *only* correct if every row really is that height; variable-height content requires a custom `VirtualScrollStrategy` that measures actual rendered row heights and maintains a running offset table, which is more expensive (must measure after render, then correct estimated positions).

4.3 How `FocusTrap` manages tabbable boundaries

1. On initialization, `cdkTrapFocus` (backed by `FocusTrapFactory` / `ConfigurableFocusTrap`) inserts two invisible, zero-size **anchor elements** (`<div cdk-focus-trap-anchor tabindex="0">`) — one immediately before the trapped content and one immediately after it, inside the DOM.
2. It attaches focus listeners to these anchors. When focus lands on the *end* anchor (meaning the user Tabbed forward past the last real tabbable element inside the trap, since focus order follows DOM order), the trap's handler immediately redirects focus to the **first** tabbable element inside the trapped region (found by walking descendants and checking each against `InteractivityChecker` — visible, not `disabled`, not `tabindex="-1"`, etc.). Symmetrically, focus landing on the *start* anchor (Shift+Tab wrapping backward past the first element) redirects to the **last** tabbable element inside.
3. This means the trap doesn't intercept every keystroke or call `preventDefault()` on Tab — it lets the browser's native Tab traversal happen naturally and merely **catches focus at the boundary and redirects it**, which is more robust across browsers/AT than manually enumerating tabindex order on every keydown.
4. `cdkTrapFocusAutoCapture` additionally calls `.focus()` on the first tabbable element (or the trap container itself if none) immediately when the trap attaches, and separately, dialog-style consumers typically capture the previously-focused element beforehand and call `.focus()` on it again when the trap/dialog is destroyed, so keyboard focus visibly "returns" to the trigger — this restore step is the consumer's/CDK-dialog's responsibility layered on top of the trap itself, not something `cdkTrapFocus` does unprompted.

---

5. Edge Cases & Gotchas

- **Overlay positioning breaks silently without the right scroll strategy.** If you create a `FlexibleConnectedPositionStrategy`-based overlay with the default `noop()` scroll strategy (or forget to set one), the overlay will not follow its trigger when an ancestor scrolls — it visually "detaches" and floats in the wrong place. For any trigger-anchored overlay (tooltip, menu, autocomplete), you almost always want `reposition()`; for modal dialogs you usually want `block()`. This is one of the single most common CDK bugs in production apps.
- `reposition()` **without throttling can thrash.** Every scroll event triggers a synchronous re-measure +

re-position; on a fast trackpad/momentum scroll this can cause jank. Always pass `{ scrollThrottle: <ms> }`.

- **Overlay content clipped by `overflow: hidden` — except it shouldn't be, and if it still is, you likely used the wrong attach mechanism** (e.g., manually appending DOM instead of using `Overlay`), or you've set `hasBackdrop`/`panelClass` styles that accidentally constrain the pane, or the *page itself* has `overflow: hidden` on `<body>`/`<html>` which also clips the overlay container (since it's a `<body>` child) — a `block()` scroll strategy or 100vh layout trap can cause this.
- **Virtual scroll assumes fixed, accurate `itemSize`.** If actual rendered row height differs from the declared `itemSize` (e.g., text wraps to two lines sometimes, or content is loaded async and changes height after render), the scrollbar thumb size/position and the index-to-offset math become wrong — rows can visually overlap or leave gaps. Either guarantee truly fixed height with CSS (`overflow: hidden`, fixed line-height) or implement a custom `VirtualScrollStrategy` for variable heights.
- **Virtual scroll + non-CDK scrolling ancestor mismatches.** The viewport must itself be the scrolling element (`cdk-virtual-scroll-viewport` sets its own `overflow: auto`); wrapping it in another scrollable ancestor, or trying to let the *page* scroll while expecting the viewport to virtualize, breaks the range calculation.
- **Virtual scroll and dynamic item insertion/removal at runtime** requires calling `viewport.checkViewportSize()` / letting `*cdkVirtualFor`'s bound array reference change (immutably swapping in a new array, or using the `DataSource` interface) — mutating the array in place without CDK noticing can leave the rendered range stale.
- **Drag-and-drop performance with large lists.** `cdkDropList` computes sibling displacement on every pointer-move by measuring `getBoundingClientRect()` for items in the list; with lists of hundreds/thousands of draggable items this becomes expensive per frame. Combine with virtual scrolling carefully — dragging across a virtualized list is non-trivial because items outside the rendered range don't exist in the DOM to be measured/displaced; CDK does not fully solve drag inside a virtual scroll viewport out of the box, and workarounds (custom index mapping, disabling in-list animation, `cdkDropListAutoScrollDisabled` tuning) are often needed.
- **Drag-and-drop only updates the DOM/visual order — never your data model.** Forgetting to call `moveItemInArray`/`transferArrayItem` (or your own equivalent) in the `(cdkDropListDropped)` handler leaves the visual reorder unsynced with your actual array; a subsequent change-detection cycle can visually "snap back."
- **Missing a11y wiring in custom components built on CDK primitives.** Using `CdkOverlay`/`Portal` to build a custom dropdown does **not** automatically give you `role="menu"`/`role="listbox"`, `aria-expanded`, `aria-activedescendant`, or keyboard arrow-key navigation — those are your responsibility to add; CDK gives you positioning/focus-trap/live-announcer *primitives*, not a finished accessible widget. Similarly, `cdkTrapFocus` traps Tab-order but does not set `role="dialog"`/`aria-modal="true"` for you, and `LiveAnnouncer` only helps if you actually call `.announce()` at the right state-change moments.
- `LiveAnnouncer` **message collision.** Calling `announce()` rapidly in succession (e.g., on every keystroke of a live filter) can cause screen readers to skip or garble overlapping announcements; debounce/throttle announcements tied to fast-changing state.
- **Overlay `panelClass`/global styles leaking.** Because all overlay panes live in one shared `cdk-overlay-container` at the `<body>` root, styles meant to be component-scoped (Angular's `ViewEncapsulation.Emulated` scoping via `_ngcontent-*` attributes) do **not** automatically apply the same way, since the overlay content is instantiated via a `ComponentPortal`/`ViewContainerRef` in a

different DOM location than the styles' host — this typically still works because Angular attaches the same `_ngcontent` attribute to the projected view, but global/unscoped styles bleeding into all overlays via shared `panelClass` names is a common source of unexpected cross-component styling bugs.

- **BreakpointObserver and SSR.** `matchMedia` doesn't exist during server-side rendering; `BreakpointObserver` needs a browser platform check (or Angular's built-in platform guards) to avoid errors when the app is rendered on the server — typically handled automatically by Angular's SSR machinery via `isPlatformBrowser`, but custom wrappers around it should still guard.
- **Multiple `CdkTable` column defs sharing a name across different table instances** in the same view (rare, but happens with dynamic/generated tables) can cause silent column-not-found rendering because `cdkColumnDef` names are resolved by string ID.

---

6. Interview Questions & Answers

Q1. What is Angular CDK, and how does it relate to Angular Material?
CDK (Component Dev Kit) is a library of unstyled, accessible behavior primitives — overlay positioning, portals, drag-and-drop, virtual scrolling, focus management, breakpoint observation, and a headless table model. Angular Material is a fully-styled Material Design component library **built on top of** CDK: e.g. `mat-menu` uses `CdkOverlay` + `FlexibleConnectedPositionStrategy`, `mat-table` extends `CdkTable`, Material dialogs use `FocusTrap` / `LiveAnnouncer` internally. CDK has no dependency on Material and can be used entirely standalone to build a custom-styled component that still gets Material-grade behavior correctness.

Interviewer intent: Checks whether the candidate understands the layered architecture (behavior vs. presentation) rather than treating "Material" and "CDK" as interchangeable/bundled.

Q2. Why would you use `CdkOverlay` instead of just absolutely-positioning a `<div>` yourself?
Hand-rolled absolute positioning breaks in several ways CDK solves for free: ancestor `overflow:hidden` / `transform` clipping the panel (CDK renders in a shared container appended to `<body>`, escaping local stacking/clipping contexts), keeping the panel anchored to its trigger during scroll/resize (position strategies + scroll strategies), viewport-overflow handling (flip/push fallback logic in `FlexibleConnectedPositionStrategy`), consistent z-index stacking across many simultaneously-open overlays, and backdrop/click-outside/escape-key handling. Reimplementing all of that correctly and consistently across browsers is significant, easy-to-get-wrong work that CDK has already solved.

Q3. What are the three Portal types and when would you use each?
`ComponentPortal<T>` wraps an Angular component *type* — use it when the overlay content is a standalone component (e.g. a whole custom dropdown-panel component) instantiated dynamically. `TemplatePortal` wraps a `TemplateRef` + `ViewContainerRef` (+ optional context) — use it when the *caller* supplies inline template content via `<ng-template>`, common for directive-style APIs like a tooltip directive that takes the consumer's own markup. `DomPortal` wraps a raw existing DOM node/element ref directly, with no Angular component or template involved — used for moving pre-existing native DOM content into an overlay.

Q4. Explain the difference between `reposition()`, `block()`, `close()`, and `noop()` scroll strategies.
`reposition()` recalculates and updates the overlay's position on every (throttled) scroll event so it stays visually anchored to its trigger — right choice for menus/tooltips/autocomplete panels attached to an origin element. `block()` prevents the page from scrolling at all while the overlay is open (used for modal dialogs where nothing behind it should move). `close()` dismisses the overlay as soon as any scrolling starts (used by some transient pickers/menus where staying open during scroll doesn't make sense). `noop()` does nothing —

the overlay stays fixed in the viewport/detaches visually from its trigger; rarely correct for a connected overlay, occasionally fine for viewport-global overlays like a centered dialog.

Q5. How does `CdkVirtualScrollViewport` achieve performance on lists with tens of thousands of items?
It never renders all items — only the ones intersecting the current visible viewport plus a small buffer. The default `FixedSizeVirtualScrollStrategy` uses `scrollTop / itemSize` arithmetic (no DOM measurement) to compute which index range is visible, `*cdkVirtualFor` instantiates/destroys embedded views only for that range (recycling views as you scroll), and a virtual "total size" spacer keeps the native scrollbar's proportions correct for the full logical list length even though the real DOM only ever contains a handful of rows. This keeps DOM node count, layout, and paint cost constant regardless of total item count.

Interviewer intent: Distinguishes candidates who've actually used virtual scroll from those who just know the term — the follow-up "what if rows have variable height" separates people who understand the mechanism from people who memorized the feature name.

Q6. What happens if actual row height doesn't match the configured `itemSize` in a virtual scroll viewport?
The index↔scrollTop math the default fixed-size strategy relies on becomes wrong: rows can visually overlap, leave gaps, or the scrollbar thumb size/position becomes inaccurate, since the strategy assumes `itemSize` uniformly and never measures actual rendered heights. To handle genuinely variable-height rows correctly you need a custom `VirtualScrollStrategy` that measures rendered heights and maintains a running offset table — more expensive since it requires post-render measurement/correction rather than pure arithmetic.

Q7. Does `cdkDropList` / `cdkDrag` mutate your data array automatically when an item is dropped?
No. `(cdkDropListDropped)` gives you a `CdkDragDrop<T>` event with `previousIndex` / `currentIndex` /containers, but CDK only handles the *visual* drag interaction (using CSS transforms, not DOM reflow, for smooth 60fps dragging) — you are responsible for updating your actual data model, typically via the provided `moveItemInArray()` helper for same-list reorders or `transferArrayItem()` for cross-list drags. If you forget, the DOM visually reorders during the drag animation but then reverts/desyncs from your unchanged array on the next render.

Q8. How would you make a custom draggable list performant with thousands of items?
Plain `cdkDropList` measures sibling `getBoundingClientRect()` s on every pointer move to compute displacement, which scales poorly with very large lists. Practical mitigations: combine with virtual scrolling (though dragging *within* a virtualized viewport is not fully solved out-of-the-box by CDK since off-screen items don't exist in the DOM to measure/displace — often requires custom index-mapping logic), limit the draggable/droppable scope to a visible "working set," disable sibling-sorting animation if not essential, and tune `cdkDropListAutoScrollDisabled` / `cdkDropListAutoScrollStep` to control auto-scroll-on-drag-near-edge cost.

Q9. What does `FocusTrap` actually do mechanically, and what does it not do?
It inserts invisible anchor elements before/after the trapped content and, when focus lands on either anchor (meaning native Tab traversal walked past the first/last tabbable element), redirects focus back to the opposite end of the trapped region — it lets native Tab order happen and only intercepts at the boundaries, rather than intercepting every keydown. It does **not** automatically add `role="dialog"` / `aria-modal="true"`, does not move initial focus into the trap unless you use the `cdkTrapFocusAutoCapture` variant, and does not restore focus to the previously-focused trigger element on destroy — that's the consumer's (or CDK Dialog's) responsibility.

Interviewer intent: Probes whether the candidate has actually built an accessible modal and knows FocusTrap is a piece of the puzzle, not a complete accessible-dialog solution.

Q10. What's the difference between `FocusMonitor` and plain CSS `:focus-visible`?
`FocusMonitor` (via `cdkMonitorFocus`) determines the *origin* of a focus event — `'mouse'`, `'keyboard'`, `'touch'`, or `'program'` (via a `.focus()` call) — by tracking the most recent user input event type before the focus event fired, and exposes this as an Observable plus CSS classes (`cdk-focused`, `cdk-keyboard-focused`, `cdk-mouse-focused`, etc.) applied to the element. This predates reliable cross-browser `:focus-visible` support and gives Angular components (and app authors) explicit, scriptable access to the focus-origin distinction — useful for logic beyond styling, e.g. auto-opening a menu only on keyboard focus, not mouse click-through. Native `:focus-visible` today covers much of the same *styling* use case with less code, but `FocusMonitor` still offers programmatic access and works uniformly across the browser matrix Angular supports.

Q11. Why does `LiveAnnouncer` exist — isn't ARIA `aria-live` on the component enough?
`aria-live` regions only reliably trigger screen-reader announcements when the *content of an already-live region* changes; many dynamic UI updates (e.g., a toast that mounts and unmounts, or a value update in a region that wasn't marked live ahead of time) don't naturally get announced, especially if the changing element itself gets added/removed rather than mutated in place. `LiveAnnouncer` maintains a persistent, hidden, always-present `aria-live` element and writes announcement text into it programmatically (`announcer.announce(msg, 'polite' | 'assertive')`), giving a reliable, centrally-managed mechanism for "tell screen reader users this happened" independent of where in the DOM the triggering change occurred.

Q12. How does `BreakpointObserver` differ from just calling `window.matchMedia` yourself?
It's a thin Angular-idiomatic wrapper: `observe(query | query[])` returns an `Observable<BreakpointState>` that integrates correctly with Angular's change detection (updates trigger inside the Angular zone appropriately), supports observing multiple queries at once and getting a combined match result, ships a `Breakpoints` constants object with Material's standard breakpoint definitions so you don't hand-write media-query strings, and centralizes subscription/cleanup instead of manually wiring `matchMedia` listeners and remembering to remove them.

Q13. What is `CdkTable`, and what does it deliberately not provide compared to `mat-table`?
`CdkTable` provides the structural/data-binding model for tabular rendering — column definitions (`cdkColumnDef`) independent of row definitions (`cdkRowDef`), a `DataSource<T>` connection point supporting arrays, Observables, or custom paginated/streamed sources, and sticky row/column support — but ships **no visual styling whatsoever**: no borders, no cell padding, no header distinction, no hover states. `mat-table` is `CdkTable` (it literally extends it) plus Material's default theme CSS and directive selector aliases. You'd reach for `CdkTable` directly when building a table with entirely custom, non-Material visual design but still wanting the column/row indirection and DataSource extensibility.

Q14. Give a concrete example of a bug caused by choosing the wrong (or no) scroll strategy.
A dropdown menu is opened via `overlay.create()` with the default/`noop()` scroll strategy inside a scrollable page. The user scrolls the page while the menu is open: the trigger button moves up visually, but the menu panel (rendered in the shared `cdk-overlay-container` at `<body>` level) stays fixed in its last computed screen position, since nothing is re-running the position strategy on scroll. The menu now visually "floats away" from its trigger. The fix is `scrollStrategy: overlay.scrollStrategies.reposition({ scrollThrottle: 20 })`, which recomputes position on each scroll tick and keeps the panel anchored, or `close()` if dismissing on scroll is acceptable UX instead.

Interviewer intent: Wants a specific, mechanistic failure scenario (not just a definition) to confirm the candidate has actually debugged an overlay in production, not merely memorized the API surface.

---

7. Quick Revision Cheat Sheet

- **Philosophy:** CDK = headless/unstyled behavior primitives; Material = CDK + Material Design styling on top. Material depends on CDK, never the reverse.
- **Overlay:** `Overlay.create(config)` → `OverlayRef`; shared `cdk-overlay-container` appended to `<body>` (escapes ancestor clipping, centralizes z-index stacking).
- **Position strategies:** `FlexibleConnectedPositionStrategy` (origin-relative, flip/push fallback) for trigger-anchored panels; `GlobalPositionStrategy` for viewport-relative (e.g. centered dialogs).
- **Scroll strategies:** `reposition()` (follow trigger, needs throttle), `block()` (freeze page scroll — modals), `close()` (dismiss on scroll), `noop()` (do nothing — rarely right for connected overlays).
- **Portals:** `ComponentPortal` (component type), `TemplatePortal` (TemplateRef + ViewContainerRef, caller-supplied markup), `DomPortal` (raw DOM node). `OverlayRef` implements `PortalOutlet`.
- **Drag-drop:** `cdkDrag` (draggable, CSS-transform based), `cdkDropList` (drop zone, auto in-list sort), `cdkDropListConnectedTo` (cross-list), `moveItemInArray` / `transferArrayItem` (you must update your own data model — CDK never mutates it for you).
- **Virtual scroll:** `cdk-virtual-scroll-viewport` + `*cdkVirtualFor`; `itemSize` required by default `FixedSizeVirtualScrollStrategy`; only visible+buffered rows exist in DOM; scrollTop/itemSize arithmetic (no measurement) computes rendered range; variable heights need a custom `VirtualScrollStrategy`.
- **A11y:** `FocusTrap` / `cdkTrapFocus` (anchor-element boundary redirect, native Tab flow preserved), `cdkTrapFocusAutoCapture` (also moves focus in on attach), `LiveAnnouncer` (`announce(msg, 'polite'|'assertive')`, hidden always-present live region), `FocusMonitor` (tracks focus origin: mouse/keyboard/touch/program), `InteractivityChecker`, `AriaDescriber`, `HighContrastModeDetector`.
- **Layout:** `BreakpointObserver.observe(query|query[])` → Observable; `Breakpoints` constants (Handset/Tablet/Web/etc.); zone-aware wrapper over `matchMedia`.
- **Table:** `CdkTable` = structural row/column/DataSource model, zero styling; `cdkColumnDef` independent of `cdkRowDef` (reusable/reorderable columns); `mat-table extends CdkTable` + Material CSS.
- **CDK vs Material one-liner:** reach for CDK directly when you need Material-grade behavior correctness with a non-Material visual design.
- **Most common real-world bug:** missing/incorrect scroll strategy on a connected overlay → panel detaches visually from its trigger on scroll.
- **Second most common bug:** forgetting accessible semantics (roles, aria-*, keyboard nav, focus restore) when building custom widgets on top of CDK primitives — CDK gives you the mechanics, not a finished accessible component.

Created By - Durgesh Singh